

# Chapter 1

## Lecture 1-4 Notes

ECEN-743 Dr. Dileep Kalathil MW 4:10 PM - 5:25 PM BLOC 113

### 1.1 Introduction – 1/12

We define machine learning as a set of methods that can automatically detect patterns in data, and then use the uncovered patterns to predict future data, or to perform other... - Murphy

Supervised learning maps an input to an output based on example input-output pairs (labelled data)

Examples: image classification, email spam filtering, speech recognition

Unsupervised learning is the task of inferring a function that describes the structure of "unlabelled" data

Examples: clustering, dimensionality reduction (e.g. PCA, autoencoder)

Reinforcement learning is an area of ML concerned with how intelligent agents ought to take actions in an environment in order to maximize the notion of cumulative reward

Reinforcement learning challenges: no prior data (RL agent generates data, by taking action in an unknown environment), data generated is not i.i.d, no supervisor (optimal policy has to be learnt from the rewards observed in the self-generated data), actions have long term consequences (rewards are often delayed), dynamical systems issues (feedback and stability)

AlphaGo

### 1.2 Lecture 4 - Jan 28

#### 1.2.1 Q-value function

Value function of policy  $\pi$  is defined as

$$V_{\pi}(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s]$$

Action-value function for policy  $\pi$  is defined as

$$Q_\pi(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a\right]$$

It is easy to show

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_\pi(s')$$

this is the instantaneous reward for  $(s, a)$  and the expected value of the next state.

### 1.3 Value Policy Iteration

#### Fixed point

Let  $f : \mathcal{U} \rightarrow \mathcal{U}$ .  $u^*$  is a fixed point of  $f(\cdot)$  if  $f(u^*) = u^*$ .

Let  $f : \mathcal{U} \rightarrow \mathcal{U}$  and let  $\|\cdot\|$  be a norm defined on  $\mathcal{U}$ . We say  $f(\cdot)$  is a contraction mapping (or simply, contraction) if  $\|f(u) - f(v)\| \leq \alpha \|u - v\|$  for an  $\alpha \in (0, 1)$  and for all  $u, v \in \mathcal{U}$ .

Banach Fixed Point Theorem.

$$\|u_{n+1} - u_n\| \leq \alpha^n \|u_1 - u_0\|$$

converges when  $\alpha \leq 1$ . We can show that the sequence  $\{u_0, u_1, u_2, \dots\}$  is a Cauchy sequence. Since  $\mathcal{U}$  is closed and bounded, the sequence converges, i.e., there exists a limit  $u^* = \lim_{k \rightarrow \infty} u_k$ . Since  $f(\cdot)$  is a contraction, it is also a continuous function. Then,  $u^* = \lim_{k \rightarrow \infty} u_k = f(\lim_{k \rightarrow \infty} u_k) = f(u^*)$ .

Proof of uniqueness: suppose there exists another fixed point  $\bar{u}^* \neq u^*$ . Then,

$$\|u^* - \bar{u}^*\| = \|f(u^*) - f(\bar{u}^*)\| \leq \alpha \|u^* - \bar{u}^*\|.$$

This is a contradiction, so  $u^* = \bar{u}^*$ .

### 1.4 Bellman Optimality Equation

Let  $V^*$  be the optimal value function. Then,  $V^*$  satisfies the equation

$$V^*(s) = \max_a \left( r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right)$$

Suppose the current state is  $s$  and we take action  $a$ . From the next state onwards, we follow the optimal policy  $\pi^*$ . What is the 'value' of state  $s$  (expected cumulative discounted reward) under this procedure?

$$V(s) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} [V^*(s')]$$

reward for  $(s, a)$  plus *gamma* times the expected value of the next state under the optimal policy  $\pi^*$ . The best action we can take in the current state  $s$  is the action which maximizes the value

$V(s)$ . The 'value' of state  $s$  if we take this action is

$$\max_a (r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V^*(s')])$$

## 1.5 Value Iteration

Bellman Operator  $T : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$  is defined as

$$(TV)(s) = \max_a (r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V(s')])$$

Aim to design feedback, with  $V_{k+1} = TV_k$ , for which we want to prove converge.

Proposition: Bellman operator  $T$  is a contraction with respect to  $\|\cdot\|_\infty$ , i.e., for any  $V_1, V_2 \in \mathcal{R}^{|S|}$ ,  $\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$ .

A useful result for the proof. Let  $g, h$  be any two real-valued function. Then,

$$|\max_a g(x, a) - \max_a h(x, a)| \leq \max_a |g(x, a) - h(x, a)|$$

Proof (of contraction property)

$$\begin{aligned} |(TV_1)(s) - (TV_2)(s)| &= |\max_{a \in \mathcal{A}} (r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_1(s')]) - \max_{a \in \mathcal{A}} (r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_2(s')])| \\ &\leq \max_{a \in \mathcal{A}} |(r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_1(s')]) - (r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_2(s')])| \\ &= \max_{a \in \mathcal{A}} |\gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_1(s')] - \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V_2(s')]| \\ &\leq \max_{a \in \mathcal{A}} \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[|V_1(s') - V_2(s')|] \\ &\leq \max_{a \in \mathcal{A}} \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[\|V_1 - V_2\|_\infty] = \gamma \|V_1 - V_2\|_\infty \end{aligned}$$

this implies

$$\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

## Chapter 2

# Lecture 6-10 Notes

2.1 February 2, 2025

## Chapter 3

# Deep Q-Learning

Feb. 23

Similar to homework questions. Some proof, think-through questions. Mix of intuition, understanding, and some mathematical details. Typically one challenging question at the end of the exam. Up to and including Deep Q-Learning.

### 3.1 Experience Relay

Put all data in a buffer and randomly sample.

### 3.2 Target Network

Q-learning gradient update:

$$w = w + \alpha(r_k + \gamma \max_b Q(w)(s_{k+1}, b) - Q(w)(s_k, a_k)) \nabla_w Q(w)(s_k, a_k)$$

In supervised learning, the target does not depend on the parameter. In Q-learning, target  $r_k + \gamma \max_b Q(w)(s_{k+1}, b)$  depends on  $w$ , so the target is moving as the parameter is updated.

### 3.3 DQN Algorithm

### 3.4 Maximization Bias in Q-Learning

Let  $\hat{\pi} = \arg \max_a \hat{Q}(s, a)$  and  $\hat{V}_{\hat{\pi}} = \max_a \hat{Q}(s, a)$ .

$$\mathbb{E}[\hat{V}_{\hat{\pi}}] = \mathbb{E}[\max_a \hat{Q}(s, a)] \geq \max_a \mathbb{E}[\hat{Q}(s, a)]$$

The greedy policy obtained from the estimated  $Q$  values can yield a maximization bias when the number of samples are finite. Maximization bias is DQN, from [Van Hasselt et al., 2016].

### 3.5 Double Estimator

Consider two random variables  $Y^1$  and  $Y^2$  with mean values  $\mu^1$  and  $\mu^2$ . Let  $\mathcal{Y}^1$  and  $\mathcal{Y}^2$  be the sets of i.i.d. samples of  $Y^1$  and  $Y^2$ . Our goal is to estimate  $\max_i \mu^i$ . With a single estimator,

we let  $\hat{\mu}^i = \frac{1}{|\mathcal{Y}^i|} \sum_{y \in \mathcal{Y}^i} y$  and let  $\hat{i}^* = \arg \max_i \hat{\mu}^i$ . Then, single estimator is  $\hat{\mu}_{\max} = \hat{\mu}^{\hat{i}^*}$ . This single estimator gives an overestimate because we are using the same samples both to determine the maximizer and value.

Possible solution is to divide the samples into two sets and use them to learn two independent estimates. Now we split  $\mathcal{Y}^i$  into  $\mathcal{Y}_A^i$  and  $\mathcal{Y}_B^i$  and then compute  $\hat{\mu}_A^i = \frac{1}{|\mathcal{Y}_A^i|} \sum_{y \in \mathcal{Y}_A^i} y$  and  $\hat{\mu}_B^i = \frac{1}{|\mathcal{Y}_B^i|} \sum_{y \in \mathcal{Y}_B^i} y$ . Now find  $\hat{i}^* = \arg \max_i \hat{\mu}_A^i$  and get double estimator  $\hat{\mu}_{\max} = \hat{\mu}_B^{\hat{i}^*}$ . We can show that the double estimator is an underestimator, i.e.,  $\mathbb{E}[\hat{\mu}_{\max}] \leq \max_i \mathbb{E}[Y^i]$ .

*Proof.* Let  $i^* = \arg \max_i \mathbb{E}[Y^i]$  be the true maximizer. Then,

$$\begin{aligned} \mathbb{E}[\hat{\mu}_{\max}] &= \mathbb{E}[\hat{\mu}_B^{\hat{i}^*}] = \mathbb{P}(\hat{i}^* = i^*) \mathbb{E}[\hat{\mu}_B^{\hat{i}^*} | \hat{i}^* = i^*] + \mathbb{P}(\hat{i}^* \neq i^*) \mathbb{E}[\hat{\mu}_B^{\hat{i}^*} | \hat{i}^* \neq i^*] \\ &= \mathbb{P}(\hat{i}^* = i^*) \max_i \mathbb{E}[Y^i] + \mathbb{P}(\hat{i}^* \neq i^*) \mathbb{E}[\hat{\mu}_B^{\hat{i}^*} | \hat{i}^* \neq i^*] \\ &\leq \mathbb{P}(\hat{i}^* = i^*) \max_i \mathbb{E}[Y^i] + \mathbb{P}(\hat{i}^* \neq i^*) \max_i \mathbb{E}[Y^i] = \max_i \mathbb{E}[Y^i] \end{aligned}$$

□

### 3.6 Double DQN

Use one Q-network to select the max-action and another Q-network to evaluate it. DQN maintains two networks anyway (one as the target network and one as the current network)

$$w = w + \alpha(R + \gamma \max_b Q(w^-)(s', b) - Q(w)(s, a)) \nabla Q(w)(s, a)$$

where  $w^- = w$ , after every  $N$  steps. To implment double DQN, use the current Q-network to select the max-action, and the target Q-network to evaluate that action

$$w = w + \alpha(r + \gamma Q(w^-)(s', \arg \max_b Q(w)(s', b)) - Q(w)(s, a)) \nabla Q(w)(s, a)$$

#### 3.6.1 Advantage Function

The advantage function is defined as

$$A_\pi(x, a) = Q_\pi(x, a) - V_\pi(x)$$

The value function measures how good it is to be in a particular state; the Q function measures the value of choosing particular action when in that state; the advantage function obtains a relative measure of the importance of each action.

#### 3.6.2 Dueling DQN Architecture

In DQN, there is a single stream to estimate Q function. In DDQN, two streams separately estimate (scalar) state-value and the advantages for each action. These two streams are particularly useful in states where its actions do not affect the environment in any relevant way. Dueling architecture can

more quickly identify the correct action during policy evaluation as redundant or similar actions are added to the learning problem.

### 3.6.3 Prioritized Experience Replay (sampling from Replay Buffer)

Some experience (data samples  $(s_i, a_i, r_i, s_{i+1})$ ) may be more informative than other. In experience relay, samples are taken uniformly at random from the replay buffer. Uniform sampling will not be able to make use of such informative samples efficiently.

Can we prioritize the samples to accelerate the learning progress? Intuition: prioritize a sample based on how much we can learn from a transition (expected learning progress). Proxy for this is TD error: for  $e_i = (s_i, a_i, r_i, s_{i+1})$ , TD error is defined as

$$\delta_i = r_i + \gamma \max_b Q(w)(s_{i+1}, b) - Q(w)(s_i, a_i)$$

In prioritized experience replay, sample  $e_i$  is selected with probability  $p_i$ ,

- Proportional sampling:  $p_i = \frac{|\delta_i|^c}{\sum_i |\delta_i|^c}$
- Rank-based sampling:  $p_i = \frac{1}{\text{rank}(e_i)}$   
where the rank of  $e_i$  when the repla memory is sorted according to  $|\delta_i|$ .

Priorities experience replay offers a performance improvement over DQN.

### 3.6.4 Rainbow DQN

The deep reinformcent learning community has made several independent improvements to the DQN algorithm. However, it is unclear which of these extensions are complementary and can be fruitfully combined. Hessel et al., 2018, shows that the combination provides state-of-the-art performance on the Atari 2600 benchmark, both in terms of data efficiency and final performance.

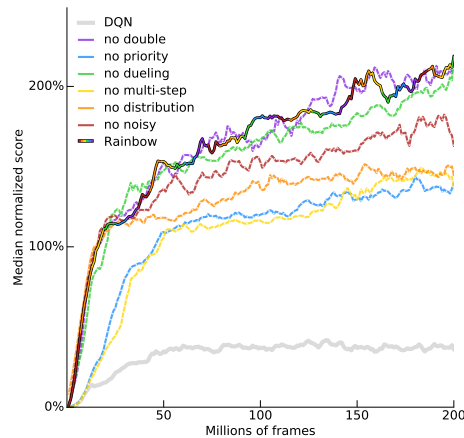


Figure 3.1: Median human-normalized performance

Later on, Agent57 was the first deep reinformcent learning agent to obtain a score that is above the human baseline on all 57 Atari 2600 games.